Assignment 2: Application Development and Database Index Structure Implementation

1. Project Objective

The objective of this assignment is to move from the database design phase (Assignment 1) into active implementation through two independent modules.

In **Module A,** you will dive into low-level data structures by building a lightweight Database Management System (DBMS) indexing engine from scratch.

In **Module B,** you will focus on high-level application logic by developing a secure, optimised local web application with a Role-Based Access Control (RBAC) API layer.

Core Technical Pipeline:

Module A: Lightweight DBMS with B+ Tree Index.

- **Core Data Structure Implementation:** Implementing the B+ Tree logic from scratch, ensuring correct node splitting, merging, and leaf-node linkage for exact and range queries.
- **DBMS Integration & Value Storage:** Building the "Table" abstraction to associate stored values (records) with keys in your tree, and managing these structures via a custom Database Manager.
- **Performance Benchmarking & Visualisation:** Conducting a rigorous comparative analysis (B+ Tree vs. Brute Force approach), generating performance graphs, and visualising the tree structure using Graphviz.

Module B: Local API Development, RBAC, and Database Optimization.

- **Local Database & UI Setup:** Initialising a local database environment for your project-specific tables and developing a web-based Member Portfolio UI.
- **Secure API Integration & RBAC:** Developing local REST APIs that handle CRUD operations, validate sessions locally, enforce strict Role-Based Access Control (Admin vs. Regular User), and maintain security audit logs.
- **SQL Indexing & Profiling:** Applying logical SQL indexing to your project tables to optimise query execution, measuring performance benchmarks

Deadline

6:00 PM, 22 March 2026

Instructor: Dr. Yogesh K. Meena

March 7, 2026 • Semester II (2025 - 2026) • CS 432 – Databases(Course Project/Assignment 2)

1. Assignment Overview

This assignment consists of **Two modules (Module A and Module B)**.

- Module A: Lightweight DBMS with B+ Tree Index (The "Engine")
- Module B: API Development & Security (The "Interface")

2. Module A: Lightweight DBMS with B+ Tree Index.

You are required to implement a B+ Tree from scratch in Python. This structure will serve as the indexing engine for your database, supporting efficient insert, update, delete, select, aggregation, and range queries. You will also conduct a performance analysis comparing your B+ Tree against a "Brute Force" linear approach.

Introduction

Efficient data storage and retrieval are fundamental challenges in computer science, particularly in database systems and file indexing. The B+ Tree is a self-balancing tree structure that improves performance in both disk- and memory-based data management. It optimises search, insertion, and deletion operations, making it widely used in database indexing, file systems, and key-value stores. This assignment focuses on implementing a B+ Tree with essential operations, including insertion, deletion, search, and range queries. Furthermore, a performance analysis is conducted by comparing the B+ Tree with a brute-force approach. This will provide insights into how structured indexing improves efficiency.
For More Understanding of the B+ tree and its implementation, this YouTube playlist might be helpful:**B+ Trees**

Implementation Details

Student should create their own separate database for module A

B+ Tree Features

Implement a B+ Tree with the following functionalities:

- **Insertion:** Keys are inserted while ensuring automatic node splitting.
- **Deletion:** Keys are removed with proper merging and redistribution.
- **Exact Search:** Ability to find whether a key exists in the tree.
- **Range Queries:** Retrieve all keys within a given range.
- **Value Storage:** Associate values (records for the table) with keys in the tree.

Performance Analysis

Compare the B+ Tree with the `BruteForceDB` approach by measuring:

- **Insertion Time:** How long it takes to insert keys in both structures.
- **Search Time:** Compare the time taken for exact match searches.
- **Deletion Time:** Compare the time taken to delete the records.

- **Range Query Time:** Measure the efficiency of retrieving keys in a range.
- **Random Performance:** Performance of random tasks (Insertion, Search, Deletion).
- **Memory Usage:** Track how much memory is used by each structure.
- **Automated Benchmarking:** Conduct multiple tests and collect results.

Visualization

Use Graphviz to visualise the tree structure:

- **Tree Structure:** Show the hierarchy of internal and leaf nodes.
- **Node Relationships:** Display parent-child relationships.
- **Leaf Node Linkage:** Highlight linked list connections in leaves.

Implementation Tasks

File Structure

```
db_management_system/

 database/
    __init__.py
    db_manager.py              # Database manager
    table.py                   # Table implementation
    bplustree.py               # B+ Tree implementation
    bruteforce.py              # BruteForceDB

 report.ipynb                   # Report and Visualizations
 requirements.txt               # Project dependencies
```

SubTasks

SubTask 1: Implement the B+ Tree

- Define `BPlusTreeNode` and `BPlusTree` classes.
- Implement insertion, deletion, search, and range queries.
- Ensure automatic splitting and merging of nodes.

SubTask 2: Implement Performance Analysis

- Create the `PerformanceAnalyzer` class.
- Implement methods to measure time complexity and memory usage.
- Compare the B+ Tree with the brute-force approach (`BruteForceDB`).

SubTask 3: Implement Visualisation

- Use Graphviz (`graphviz.Digraph`) to visualise the tree.
- Implement node connections and leaf node linkages.

SubTask 4: Conduct Performance Testing

- Generate different sets of random keys (e.g., using `range(100, 100000, 1000)`).
- Insert random key sets of different sizes into both data structures.
- Again, generate random key sets, perform search operations, and measure execution time.
- Do the same for range queries and delete operations; compare efficiency.
- Visualise the results using Matplotlib plots.

SubTask 5: Video Demonstration with Audio Explanation

Record a short screen-capture video (3–5 minutes) demonstrating your working B+ Tree implementation and performance analysis. **You must include a clear audio voice-over/explanation** that walks us through your project. Your video and audio explanation must clearly cover:

- **Code & Functionality:** Briefly show your B+ Tree structure and demonstrate the core operations (insertion, deletion, search, and range queries) executing successfully in your notebook or terminal.
- **Visualisation:** Show the Graphviz visualisations of your tree, highlighting how nodes split or merge and how leaf nodes are linked.
- **Performance Analysis:** Walk us through the Matplotlib graphs and benchmarking tables you generated. Explain why the data looks the way it does (e.g., explaining the time complexity difference between your B+ Tree and the BruteForceDB during range queries or large inserts).

Note: Host the video on a platform like Google Drive or YouTube (Unlisted) and include the link in your report.ipynb file.

SubTask 6: Write a Report

Test the database created in a Jupyter notebook and document the following:

- **Introduction:** Explain the problem addressed and the proposed solution (B+ Tree DBMS).
- **Implementation:** Describe the implementation details of the B+ Tree operations.
- **Performance Analysis:** Present the benchmarking results using tables and graphs. Discuss the findings.
- **Visualisation:** Include results of the generated tree visualisations of your database tables.
- **Conclusion:** Summarise the project findings, challenges, and potential future improvements.

Evaluation Criteria

Criteria	**Marks**
Implementation B+ Tree (insertion, deletion, search, range queries, value storage)	20
Automated benchmarking and performance analysis (time/memory comparison)	10
Visualisation using Graphviz Report and Video	10
Total	**40**

3. Module B: Local API Development, RBAC, and Database Optimisation

In this module, you will focus on bringing the project-specific database schemas you designed in Task 1 to life. You will develop a web UI and secure APIs to interact with your tables, manage core system data locally, and implement strict Role-Based Access Control (RBAC). Finally, you will apply SQL indexing to optimise your database access and quantitatively measure the performance improvements of your queries. You are free to choose your preferred tech stack, local server environment, and directory structure for this assignment.

Prerequisites

1. **Programming:** Proficiency in your chosen backend language (e.g., Python, Node.js, Java) to implement RESTful APIs and a frontend web UI.
2. **Database Security:** Understanding of authentication, session management, and Role-Based Access Control (RBAC).
3. **Database Optimisation:** Understanding of SQL indexing strategies (e.g., B+ Trees, Hash indexes) and query execution profiling.

SubTasks

SubTask 1: Local Environment Setup Data Management

Initialise your local database environment. You must design a system that handles both your project-specific tables (from Task 1) and core system data (which may include tables for members, user logins/credentials, and member-to-group mappings, etc, based on your project).

- **Member Creation/Deletion:** When managing members, ensure corresponding entries in your login and group mapping tables are handled correctly to maintain strict data integrity. Do not duplicate core data (like user credentials) inside your project-specific tables.

SubTask 2: API and UI Development

Develop a web-based UI and the underlying local APIs to perform CRUD (Create, Read, Update, Delete) operations on your project-specific tables.

- Ensure every API call validates the user's session using a local authentication mechanism that you develop.
- **Member Portfolio:** Create a portfolio feature in the UI that displays relevant details of members within the project. Restrict access so that only authenticated users can view this, and users can only view profiles they have permission to see.

SubTask 3: Role-Based Access Control (RBAC)

Implement strict role-based access control on your new UI and APIs:

- **Admins:** Should have full access to perform administrative actions (e.g., adding/deleting records, managing group members across the core tables).
- **Regular Users:** Should have restricted access (e.g., read-only access, or only the ability to modify their own specific portfolio records).

- Whenever an API call modifies data in your database, ensure the changes are logged locally to a file (e.g., audit.log) or a dedicated logging table.
- Any direct database modifications made without passing through your session-validated APIs should be easily identifiable as unauthorized when reviewing these logs.

SubTask 4: SQL Indexing and Query Optimisation

Identify the most frequently accessed or slowest-performing API endpoints in your application.

- Apply appropriate SQL indexing (e.g., single-column, composite, or unique indexes) to your project-specific tables to optimize data retrieval.
- Ensure your indexing strategy directly targets the WHERE, JOIN, or ORDER BY clauses used in your API's SQL queries.

SubTask 5: Performance Benchmarking

Quantitatively measure the impact of your SQL indexing:

- Record the API response times and the SQL query execution times before applying your indexes.
- Record the same metrics after applying your indexes.
- Use query profiling tools (e.g., the EXPLAIN statement) to document how the database engine's access plan changed (e.g., shifting from a full table scan to an index seek).

SubTask 6: Video Demonstration with Audio Explanation

Record a short screen-capture video (3–5 minutes) demonstrating your fully integrated local system. **You must include a clear audio voice-over/explanation** that walks us through your implementation. Your video and audio explanation must clearly cover:

- **UI & API Functionality:** Narrate your navigation of the web UI, viewing the Member Portfolio, and successfully performing CRUD operations on your Task 1 tables.
- **RBAC Enforcement:** Explain how your roles are set up while logging in as an Admin (demonstrating administrative access) and then as a Regular User (demonstrating restricted/read-only access).
- **Security Logging Check:** Briefly explain your logging mechanism while showing your local logs capturing valid API operations and highlighting any flagged unauthorised database modifications.

Note: Host the video on a platform like Google Drive or YouTube (Unlisted) and include the link in your report.

Evaluation Criteria

Criteria	Marks
Local DB & Secure API Implementation (Core/Project setup, CRUD APIs, Session Validation, Member Portfolio)	20
Security & RBAC (Role enforcement, proper deletion integrity, unauthorised access logging)	10
Database Optimization (Logical indexing strategy, query profiling, benchmarking)	10
Optimization Report (Benchmarking graphs, EXPLAIN plan analysis, documentation)	10
Video Submission & Audio Explanation (Clear narration and demonstration of UI/API functionality, RBAC constraints, and logging)	10
Total	**60**

4. Submission Guidelines (Both modules)

You are required to submit a single private GitHub repository link containing all the files for both modules. Please organize your repository clearly with separate folders for Module A and Module B.
Repository Structure: Ensure your repository is structured logically.
Example:

```
CS432_Track1_Submission/

 Module_A/
    database/ (Contains bplustree.py, bruteforce.py, etc.)
    report.ipynb (Benchmarking and Visualization)
    requirements.txt

 Module_B/
     app/ (Contains API code, UI templates, auth logic)
     sql/ (Contains database creation scripts)
     logs/ (Contains audit.log)

     report.pdf (or .ipynb)
     requirements.txt
```

Module A Deliverables:

- **Source Code:** A database folder containing all Python scripts implementing the B+ Tree, BruteForceDB, and performance analyser functionalities.
- **Jupyter Notebook Report (report.ipynb):** Must include:
  - **Introduction:** Problem statement and solution overview.

  - **Implementation Details:** Explanation of your B+ Tree logic.
  - **Performance Analysis:** Tables and Matplotlib graphs comparing B+ Tree vs. BruteForceDB for insertions, deletions, searches, and range queries.
  - **Visualisations:** Graphviz outputs showing your tree's internal nodes, leaf nodes, and linkages.
  - **Conclusion:** Summary of findings and challenges.

Module B Deliverables:

- **Source Code Scripts:**
  - Complete source code for your web UI, local APIs, and local authentication/RBAC logic.
  - SQL scripts are used to create your local core tables and project-specific tables.
- **Security Logs:** Provide your local log files (e.g., audit.log) demonstrating session validation for API queries and highlighting flagged unauthorised database modifications.
- **Optimisation Implementation Report:** Prepare a report (PDF or Jupyter Notebook) documenting:
  - **Schema Design:** How you structured your local core tables and project-specific tables to maintain integrity without duplicating data.
  - **Security:** How your session validation prevents data leaks and how your RBAC roles (Admin vs. Regular User) are enforced.
  - **Indexing Strategy:** Which columns you indexed and why (based on your API's SQL queries).
  - **Performance Benchmarking:** Quantitative results showing query execution times before and after indexing.
- Video Demonstration: Place a clear link in your report to your 3–5 minute video demonstration. Ensure the video includes your audio explanation, demonstrates UI/API functionality, shows the Admin/Regular User RBAC constraints, and demonstrates the logging mechanism in action. Ensure the link is accessible (e.g., an Unlisted YouTube video or an open Google Drive link).

5. Appendix

A. B+ tree.py

This appendix provides the boilerplate code for the bplustree.py. There may be other functions required; implement them accordingly.

```
def search ( self, key ) :
# Search for a key in the B+ tree. Return the associated value if found, else None.
# Traverse from root to appropriate leaf node.
pass

def insert ( self , key , value ) :
"""
Insert key-value pair into the B+ tree.
Handle root splitting if necessary
Maintain sorted order and balance properties.
"""
pass

def _insert_non_full ( self , node , key , value ) :
# Recursive helper to insert into a non-full node.
# Split child nodes if they become full during insertion.
pass

def _split_child ( self , parent , index ) :
"""
Split the arentchild at the given index
For leaves:preserve the linked list structure and copy the middle key to the parent.
For internal nodes: promote the middle key and split the children
"""
pass

def delete ( self, key ) :
"""
Delete key from the B+ tree.
Handle underflow by borrowing from siblings or merging nodes.
Update the root if it becomes empty.
Return True if deletion succeeded, False otherwise.
"""
pass

def _delete ( self , node , key ) :
# Recursive helper for deletion. Handle leaf and internal nodes .
# Ensure all nodes maintain minimum keys after deletion.
pass

def _fill_child ( self , node , index ) :
# Ensure child at given index has enough keys by borrowing from siblings or merging.
```

```
43 pass
44
45 def _borrow_from_prev ( self , node , index ) :
46 # Borrow a key from the left sibling to prevent underflow.
47 pass
48
49 def _borrow_from_next ( self , node , index ) :
50 # Borrow a key from the right sibling to prevent underflow
51 pass
52
53 def _merge ( self , node , index ) :
54 # Merge child at index with its right sibling. Update parent keys
55 pass
56
57 def update ( self , key , new_value ) :
58 # Update value associated with an existing key. Return True if successful.
59  pass
60
61 def range_query ( self , start_key , end_key ):
62 """
63 Return all key-value pairs where start_key <= key <= end_key.
64 Traverse leaf nodes using the following pointers for efficient range scans.
65 """
66 pass
67
68 def get_all ( self ) :
69 # Return all key-value pairs in the tree using in-order traversal.
70 pass
71
72 def visualize_tree ( self ):
73 # Generate Graphviz representation of the B+ tree structure .
74 pass
75
76 def _add_nodes ( self , dot , node ) :
77 # Recursively add nodes to Graphviz object (for visualisation.
78 pass
79
80 def _add_edges ( self , dot , node ) :
81 # Add edges between nodes and dashed lines for leaf connections (for visualisation
82 pass
```

Listing 1: Required B+ Tree methods

B. BruteForceDB

This appendix provides the Python code for the BruteForceDB class, which serves as a baseline for performance comparison against the B+ Tree. It uses a simple list to store keys and performs operations through linear iteration.

```
1 class BruteForceDB :
2 def __init__ ( self ) :
3 self. data = []
4
5 def insert ( self key ) :
6 self. data append ( key )
7
8 def search ( self, key ) :
9 return key in self. data
10
11 def delete ( self key ) :
12 if key in self data :
13 self. data. remove ( key )
14
15 def range_query ( self , start , end ) :
16 return [ k for k in self. data if start <= k <= end ]
```

Listing 2: BruteForceDB Class.

API Documentation

`/login (POST)`
Description: Authenticates a user and issues a session token.
Arguments (JSON):

- user (string): Username.
- password (string): User's password.

Returns:

- 200 (JSON): {
  "message": "Login successful",
  session token: 'jwt session token'
  }
- 401 (JSON):
  - {"error": "Invalid credentials"}
  - {"error": "Missing parameters"}

  on failure.

`/isAuth (GET)`
Description: Checks if a user's session is valid.
Arguments:

- {
  session token: 'JWT session token'
  }

Returns:

- 200 (JSON): {"message": "User is authenticated", "username": string, "role": string, "expiry": datetime} on success.
- 401 (JSON):
  - {"error": "No session found"}
  - {"error": "Session expired"}
  - {"error": "Invalid session token"}

  on failure.

`/ (GET)`
Description: Simple welcome endpoint.
Arguments: None.
Returns:

- 200 (JSON): {"message": "Welcome to test APIs"}.